

Módulo 01 — Como LLMs funcionam

Cada slide da apresentação explicado em linguagem simples: termos, o que falar e o que responder.

1. Slide a slide

Slide a slide

Este guia explica cada slide da apresentação em linguagem simples: o que significa cada termo, o que falar e o que responder se perguntarem. Leia uma vez inteiro antes da aula; durante a aula, as notas do PPTX bastam.

Se só der tempo de entender uma coisa: um LLM olha o texto que já existe e calcula, para cada pedaço de texto possível, a chance de ele vir em seguida. Depois sorteia um pedaço, cola no fim e repete. Quase tudo na aula é consequência disso.

Slide 1 — Capa

Só abertura. Pergunte quem conseguiu rodar o `pnpm verificar` (o setup). Quem não conseguiu acompanha pela tela hoje.

LLM (*Large Language Model*, "modelo de linguagem grande"): o tipo de IA por trás do ChatGPT, do Claude, do Gemini etc. É um programa treinado com uma quantidade enorme de texto para continuar textos.

Slide 2 — Hoje

Os quatro objetivos da aula. Não leia a lista; diga algo como: *"Hoje a gente abre o capô. Quatro coisas que parecem mágica vão ganhar uma explicação mecânica."*

- **Prompt:** o texto que você manda para o modelo (a pergunta ou instrução).
- **Contexto:** tudo que o modelo recebe numa chamada: instruções + conversa + documentos.
- **Alucinar:** quando o modelo inventa algo falso com cara de verdadeiro.

Slide 3 — "Qual destas é a resposta certa?"

O que está acontecendo: o mesmo pedido ("escreva a primeira frase de uma newsletter sobre IA") foi feito 5 vezes ao modelo e veio uma resposta diferente em cada vez.

- **demo:temperatura:** o selo azul indica que existe uma demonstração que você roda no terminal (`pnpm -F @mentoria/ex01-llms demo:temperatura`).

- **qwen3:4b-instruct**: o nome do modelo que roda no seu computador via Ollama. "4b" = 4 bilhões de parâmetros (é um modelo pequeno; os grandes têm centenas de bilhões).
- **temperature 1.2**: um botão que controla o quanto o modelo "arrisca". Explicado no slide 11.

O que falar: pergunte "qual é a certa?" e deixe 2 ou 3 pessoas responderem. **Não explique ainda.** Diga que a resposta vem em 15 minutos (slide 13).

Slide 4 — A frase da aula

Um LLM é uma função que recebe tokens e devolve uma distribuição de probabilidade sobre o próximo token.

- **Token**: um pedaço de texto. Pode ser uma palavra inteira ("casa"), um pedaço de palavra ("amento") ou pontuação ("."). Slide 6 mostra exemplos.
- **Distribuição de probabilidade**: uma lista de "chances" que somam 100%. Por exemplo: depois de "A capital do Brasil é", o modelo pode dar 50% para "Bras", 49% para "***", 1% para o resto.
- **Sortear dessa distribuição**: escolher o próximo token de acordo com essas chances, como girar uma roleta em que cada fatia tem o tamanho da probabilidade.

O que falar: escreva a frase no chat. Diga que tudo o que vem depois volta para ela.

Slide 5 — Divisória "1 · Tokens"

Só marca o início do bloco. Passe direto.

Slide 6 — O modelo não lê palavras

Cada caixinha colorida é **um token**. Repare em três coisas:

1. O espaço vem grudado no começo do token ([modelo] , não [modelo]).
 2. "Parcelamento" virou dois tokens: [Parcel] [amento] .
 3. "inconstitucionalissimamente" virou cinco.
- **Tokenizador**: o programa que corta o texto em tokens. Cada família de modelos tem o seu.
 - **o200k_base**: o nome do tokenizador usado no exemplo (é o dos modelos GPT-4o da OpenAI). "200k" = tem cerca de 200 mil tokens no "dicionário".
 - **js-tiktoken**: a biblioteca JavaScript que faz esse corte no nosso código.

Por que palavras raras viram vários tokens: o tokenizador foi construído olhando muito texto e criou "atalhos" para os pedaços mais comuns. Palavra comum = um token só; palavra rara = montada com vários pedaços. A técnica se chama **BPE** (*Byte Pair Encoding*), mas você não precisa citar o nome.

Analogia para usar: um teclado que tem teclas para sílabas e palavras comuns, além das letras. "de" tem tecla própria; "inconstitucionalissimamente" precisa de várias.

Pergunta provável: "Então o modelo não sabe soletrar?" Exato. Ele vê [Parcel] [amento] , não letra por letra. Por isso LLMs erram em contar letras de uma palavra.

Slide 7 — Por que isso importa

- **Custo:** os provedores cobram por token. Mais tokens, mais caro.
- **Limite:** existe um máximo de tokens por chamada (a *janela de contexto*, slide 15).
- **Idioma:** o mesmo texto pode ocupar mais tokens em um idioma do que em outro, porque os tokenizadores foram treinados com mais texto em alguns idiomas (principalmente inglês). Um estudo mediu diferenças de até 15 vezes **em alguns casos extremos**. Não é a regra para português: no slide 6, a primeira frase deu 11 tokens em PT e 11 em EN.

Fonte no rodapé: Petrov et al. (2023) é o artigo científico que mediu isso. "et al." = "e outros autores".

Slide 8 — Divisória "2 · Um token por vez"

Passe direto.

Slide 9 — Geração autoregressiva

Autoregressiva é um nome complicado para uma ideia simples: o modelo usa o que **ele mesmo** já escreveu para decidir o próximo pedaço.

O desenho é um laço:

1. O modelo recebe o texto (o contexto).
 2. Calcula as chances do próximo token.
 3. Sorteia um.
 4. Cola esse token no fim do texto e volta ao passo 1.
- **Streaming:** quando a resposta aparece "digitando aos poucos" no ChatGPT. Não é efeito visual: ela realmente é gerada pedaço por pedaço.

- **Não há plano:** o modelo não pensa a frase inteira antes. Se ele começou mal, tende a continuar mal, porque cada token novo se apoia nos anteriores.

Analogia: o corretor do celular que sugere a próxima palavra, só que muito melhor e repetindo isso centenas de vezes seguidas.

Slide 10 — O próximo token, ao vivo

A tabela mostra o que o modelo "pensou" em cada passo ao completar *"A capital do Brasil é"*:

- No token " é", ele tinha 100% de certeza.
- No passo seguinte, ficou dividido: **50,4%** para " Bras" (de "Brasília") e **49,3%** para " **".
- ****** é o símbolo que, em *markdown* (formatação de texto usada em chats), abre um **negrito**. Ou seja, o modelo não estava em dúvida sobre a capital: estava em dúvida se escrevia "Brasília" em negrito ou não.
- **Logprobs:** abreviação de *log-probabilities*. É o recurso do provedor que devolve essas probabilidades junto com o texto. Nem todo provedor oferece.
- **temperature 0:** o modelo sempre escolhe o token mais provável (aqui, "Bras").

O que falar: "o modelo **sempre** tem uma lista de chances. Às vezes ela está concentrada (100%), às vezes dividida. Em cada passo há um sorteio."

Slide 11 — Temperatura

- **Logits:** as "notas" brutas que o modelo dá para cada token candidato, antes de virarem porcentagens. No exemplo, três candidatos com notas 1, 2 e 3.
- **Softmax:** a conta que transforma essas notas em porcentagens que somam 100%. Quanto maior a nota, maior a porcentagem.
- **Temperatura (T):** divide as notas antes da conta.
 - **T baixa (0,5):** o favorito fica ainda mais forte (86,7%).
 - **T = 1:** distribuição "normal" (66,5% para o favorito).
 - **T → 0:** o favorito leva 100%. Isso se chama **greedy** ("guloso": sempre pega o maior).
 - **T alta:** as chances ficam parecidas e os "azarões" passam a ser sorteados com frequência.

Você não precisa explicar a fórmula. A intuição basta: *temperatura é o quanto o modelo confia no favorito*. As barras do slide são os valores reais calculados.

Analogia: um sorteio com bolinhas na urna. Temperatura baixa = o favorito tem quase todas as bolinhas. Temperatura alta = as bolinhas ficam distribuídas por igual.

Slide 12 — Top-p

Outra forma de controlar o sorteio:

1. Ordena os tokens do mais provável para o menos provável.
 2. Vai somando as chances até chegar em p (por exemplo, 90%).
 3. Sorteia **só entre esses**; o resto é descartado.
- **Cauda:** os milhares de tokens com chance minúscula. Raramente são sorteados, mas quando são, geram texto sem sentido. O top-p corta essa cauda.
 - **Nucleus sampling:** outro nome para top-p. "Núcleo" é o grupo que sobra.

A citação no slide diz que escolher sempre o mais provável gera texto "sem graça e repetitivo", e sortear de tudo gera lixo. O top-p fica no meio-termo.

Dica prática para falar: mexa em temperatura **ou** em top-p, não nos dois ao mesmo tempo, senão você não sabe qual causou o efeito.

Slide 13 — Voltando ao gancho

Agora você responde a pergunta do slide 3:

- Com **temperature 0**, as 5 respostas saíram idênticas (sempre o favorito).
- Com **temperature 1.2**, saíram 5 diferentes.
- **Nenhuma é "a certa"**: o modelo não tem *uma* resposta guardada; ele tem uma distribuição de chances, e cada execução é um sorteio.

Regra prática: tarefas que precisam de consistência (classificar, extrair dados, gerar código) usam temperatura baixa. Tarefas criativas (brainstorm) usam temperatura mais alta.

O aviso ⚠ : mesmo com temperature 0, alguns provedores podem dar respostas levemente diferentes (por detalhes de hardware e de como processam várias requisições juntas). Não trate como garantia.

Slide 14 — Divisória "3 · Contexto, custo e latência"

Passe direto.

Slide 15 — Context window

- **Context window (janela de contexto):** o máximo de tokens que o modelo consegue considerar numa única chamada, somando o que você manda e o que ele responde.
- **Não tem memória entre chamadas:** cada chamada começa do zero. Quando um chat "lembra" do que você disse antes, é porque o aplicativo **reenvia a conversa inteira** a cada mensagem. Por isso conversas longas ficam mais caras.
- **Quando não cabe:** alguém precisa cortar. Pode ser o provedor (que recusa), a biblioteca (que corta automaticamente) ou o seu código (que escolhe o que tirar). No exercício, a pessoa implementa um corte que mantém as instruções e as mensagens mais recentes.
- **Lost in the middle ("perdido no meio"):** um estudo mostrou que, em textos muito longos, o modelo usa pior a informação que está no meio do que a que está no começo ou no fim.

Pergunta provável: *"Como o ChatGPT lembra de conversas antigas?"* Por recursos de memória do aplicativo, que guardam resumos e os colocam de volta no contexto. O modelo em si não lembra.

Se perguntarem sobre RAG: é a técnica de buscar só os trechos relevantes de documentos e colocá-los no contexto, em vez de mandar tudo. Aparece mais à frente na trilha.

Slide 16 — Custo e latência

Custo:

- Você paga um preço por milhão de tokens de **entrada** (o que você manda) e outro por milhão de tokens de **saída** (o que o modelo escreve). A saída costuma ser mais cara.
- O exemplo usa **preços inventados** (US\$ 2 e US\$ 8 por milhão) só para mostrar a conta: 1.500 tokens de entrada + 300 de saída = US\$ 0,0054 (meio centavo de dólar). Parece pouco, mas multiplique por milhões de chamadas.

Latência (tempo de resposta):

- **Entrada:** o modelo lê o texto de entrada todo de uma vez, em paralelo. Entrada grande atrasa um pouco o primeiro token.
- **Saída:** gerada um token por vez (slide 9). Resposta longa = resposta lenta.

Conclusão para falar: para tarefas repetidas em grande volume, peça respostas **curtas e estruturadas** (por exemplo, só `{"relevante": true}` em vez de um parágrafo). Esse é o gancho da Aula 2.

Slide 17 — Divisória "4 · Alucinação"

Passe direto.

Slide 18 — Por que o modelo inventa

- O modelo escolhe o texto **mais plausível** (que "soa certo"), não o texto **verdadeiro**. Ele não consulta uma base de fatos.
- Se você pede "resuma o artigo X" e o artigo não existe, a continuação mais plausível desse pedido é... um resumo. E ele escreve um, com o mesmo tom confiante de sempre.

A frase do quadro: "o modelo mentiu" é impreciso, porque mentir pressupõe saber a verdade e querer enganar. O modelo apenas **completou o texto**.

Slide 19 — Três perguntas sobre coisas que não existem

Os três itens foram **inventados para a demo**: o artigo "Gradientes Tropicais", o framework "Jabuticaba.js" e a cidade "Porto Esmeralda do Norte".

- **Artigo:** o modelo resumiu um artigo inexistente com detalhes técnicos. Invenção total.
- **Framework:** ele acertou ao dizer que o Jabuticaba.js não é um framework de agentes, mas inventou que é um "framework web". Invenção parcial.
- **Cidade:** ele acertou ao dizer que a cidade não existe, mas inventou um "complexo turístico em Pernambuco" para parecer útil.

Termos: *NeurIPS* é uma das maiores conferências científicas de IA (o que deixa a pergunta ainda mais "crível"). *Framework* é uma biblioteca de código que dá estrutura para construir algo. *Orquestração de agentes* é coordenar vários programas de IA trabalhando juntos (Módulo 5).

Mensagem: alucinação não é tudo-ou-nada. Mesmo quando acerta o principal, o modelo pode inventar o entorno.

Atenção: as respostas são longas. Se o tempo estiver curto, mostre o slide em vez de rodar a demo.

Slide 20 — Por que ela persiste

Resumo de um artigo de 2025 (Kalai et al.) com duas causas:

- **No pré-treino** (a fase em que o modelo aprende lendo muito texto): se nem sempre dá para distinguir uma afirmação falsa de uma verdadeira só olhando o texto, o modelo vai errar às vezes. É estatística.
- **Nas avaliações:** os testes que medem os modelos (**benchmarks**, como provas padronizadas) normalmente dão ponto para acerto e zero para "não sei". Como numa prova de múltipla escolha sem desconto por erro, **chutar compensa**. Então os modelos aprendem a

chutar.

O que fazer: o resto da trilha. Dar a fonte ao modelo, validar a resposta (Módulos 2 e 3) e mandar casos duvidosos para um humano revisar (Módulo 7).

Slide 21 — Na leitura de apoio

Três assuntos que **não** entram na aula, só uma frase de cada. Estão detalhados na apostila.

- **Embeddings:** o modelo transforma cada token numa lista de números (um **veto**r) que representa o significado. Palavras com significados parecidos ficam com números parecidos.
Analogia: um endereço num mapa de significados; "cachorro" mora perto de "gato" e longe de "imposto".
 - **Atenção:** o mecanismo pelo qual cada token "olha" os outros do texto para se entender. Em "o cliente pediu o cartão, mas **ele** foi recusado", a atenção ajuda a ligar "ele" a "cartão".
Analogia: numa reunião, antes de falar, cada pessoa decide quanto ouvir de cada colega.
 - **Pré-treino → RLHF → RLVR:** as fases do treinamento.
 - **Pré-treino:** ler a internet e aprender a continuar textos.
 - **RLHF** (*aprendizado por reforço com feedback humano*): pessoas comparam respostas e dizem qual é melhor; o modelo aprende a preferir as melhores. É o que faz ele seguir instruções em vez de só continuar o texto.
 - **RLVR** (*aprendizado por reforço com recompensas verificáveis*): em vez de uma pessoa julgar, um programa confere se a resposta está certa (a conta bate? o código passa nos testes?).
 - *Analogia: ler a biblioteca inteira → professor dizendo qual redação ficou melhor → lista de exercícios com gabarito.*
-

Slide 22 — Divisória "Discussão"

15 minutos de conversa. Você não precisa ter todas as respostas: o objetivo é a turma pensar em voz alta.

Slide 23 — Para conversar (com respostas esperadas)

1. **"Se temperature 0 dá sempre a mesma resposta, por que não usar 0 sempre?"** Porque às vezes você *quer* variedade: brainstorm, textos criativos, várias sugestões diferentes. Além disso, temperatura 0 pode deixar o texto repetitivo.
2. **"Que temperatura você usaria para classificar notícias?"** Baixa (0 ou perto). Classificação precisa de consistência: a mesma notícia deve cair sempre na mesma categoria.

3. **"Um chatbot esqueceu o começo da conversa. Por quê?"** A conversa ficou maior que a janela de contexto e o começo foi cortado; ou o aplicativo resumiu o histórico e perdeu detalhes; ou a informação ficou "perdida no meio" de um contexto longo.
4. **"O modelo mentiu. Qual seria uma descrição mais precisa?"** "O modelo gerou uma continuação plausível, mas falsa." Ele não sabe que é falso e não tem intenção.
5. **"Se saída custa mais e sai um token por vez, o que muda no prompt para alto volume?"** Pedir respostas curtas e num formato fixo (por exemplo, um JSON com campos definidos), sem explicações longas. Fica mais barato, mais rápido e mais fácil de um programa ler. É o assunto da Aula 2.

Se o tempo apertar: faça só a 2, a 4 e a 5.

Slide 24 — Para a próxima aula

Duas tarefas de casa:

- **Exercício:** implementar em código as peças que a aula mostrou (a conta da temperatura, o top-p, o sorteio e o corte de contexto). Roda sem modelo, só com testes automáticos. Quem quiser mais faz o desafio do mini gerador de frases que "alucina".
- **Projeto final, etapa M1:** um programa que pede ao modelo para dizer se uma notícia é relevante para uma newsletter de IA e em que categoria ela entra, em **texto livre**. A pessoa roda o experimento e traz o arquivo gerado.

Comandos: estão no slide. "Testes passando" é o critério de "feito". As soluções de referência estão na pasta `solucao/` de cada exercício.

Slide 25 — Próxima aula

O gancho para a Aula 2. No experimento do projeto, com temperatura alta, o modelo começou a variar o **formato** da resposta: às vezes "Relevante. Categoria: ...", às vezes "Relevantes → ...", às vezes "JUSTIFICATIVA:". Um programa que tenta ler essa resposta procurando padrões de texto (**regex**, expressões regulares) quebra quando o formato muda.

- **Decisão tipada:** em vez de texto livre, o modelo devolve dados com formato fixo e verificável, como `{"relevante": true, "categoria": "modelos"}`.
- **Confiança:** junto com a decisão, um número dizendo o quanto o modelo tem certeza (por exemplo, 0,92). Serve para mandar os casos duvidosos para revisão humana.

O que falar: "Na próxima aula a gente vai ver uma nova classe de modelos que faz exatamente isso, e como simular com o modelo que a gente já tem."

Glossário rápido

Termo	Em uma frase
LLM	IA treinada para continuar textos
Token	pedaço de texto que o modelo enxerga
Tokenizador	programa que corta o texto em tokens
Prompt	o que você manda para o modelo
Contexto / context window	tudo que o modelo recebe numa chamada / o limite disso em tokens
Distribuição de probabilidade	lista de chances que soma 100%
Logits	"notas" brutas antes de virarem porcentagens
Softmax	conta que transforma notas em porcentagens
Temperature	o quanto o sorteio favorece o favorito
Greedy	sempre escolher o mais provável
Top-p (nucleus)	sortear só entre os mais prováveis que somam p
Logprobs	as probabilidades que o provedor devolve junto com o texto
Autoregressivo	gera um pedaço, cola no texto e usa para gerar o próximo
Streaming	resposta aparecendo aos poucos
Latência	tempo de resposta
Alucinação	afirmação falsa gerada com cara de verdadeira
Benchmark	prova padronizada para comparar modelos
Embedding	lista de números que representa o significado
Atenção	como cada token "olha" os outros para se entender
Pré-treino / RLHF / RLVR	aprender lendo / aprender com notas humanas / aprender com gabarito automático
RAG	buscar trechos relevantes e colocá-los no contexto
Markdown	formatação de texto com símbolos (negrito , # título)
Regex	padrão para procurar texto num programa
JSON	formato de dados estruturado, como {"chave": "valor"}